

Agentic Video Generator

Project Overview and Status

Video Agent Team

June 14, 2026

Executive Summary

This project implements an agentic application that converts a user story into a generated video via a staged orchestration pipeline. The system currently supports persistent job tracking, asynchronous execution through a queue, ffmpeg-based assembly, and pluggable external providers for media and speech.

Current Architecture

- **API Layer:** FastAPI endpoints for job creation and job status polling.
- **UI Layer:** Streamlit frontend (Python) and React frontend (JavaScript) for submission, polling, resume action, and video preview.
- **Orchestration Layer:** LangGraph state machine with nodes: director, storyboard, asset generation, voice synthesis, editor, and critic.
- **Persistence Layer:** Postgres for durable job state and metadata.
- **Queue Layer:** Redis list queue for asynchronous job processing.
- **Checkpoint Layer:** Redis (hot cache) + Postgres (durable) checkpoints for resumable runs.
- **Render Layer:** ffmpeg pipeline for scene composition and optional narration muxing.
- **Provider Layer:** Replicate adapters for image/video generation and ElevenLabs adapter for TTS, with local fallbacks.

Implemented Features (Status: Complete)

1. FastAPI endpoints: `/generate`, `/jobs/{job_id}`, `/jobs/{job_id}/resume`.
2. Typed schemas for `VideoPlan`, `ScenePlan`, and job status models.
3. Queue-driven background execution using Redis.
4. Postgres-backed persistent job store.
5. Scene rendering with ffmpeg for image/video assets.

6. Voiceover generation path with provider abstraction.
7. Fallback behavior when provider credentials are absent.
8. LangGraph checkpointing after each successful node execution.
9. Streamlit UI with submit form, job polling, resume button, and video preview.
10. React UI (modern JavaScript-based alternative) with dark theme, responsive design, and Tailwind CSS styling.

Current Readiness and Unblocker

- Core platform is operational for local fallback mode.
- Primary unblocker for production-like output quality is provider credential setup.
- Required keys to enable real generation providers:
 - `REPLICATE_API_TOKEN` for image/video generation.
 - `ELEVENLABS_API_KEY` for TTS narration.
- `OPENAI_API_KEY` is optional in the current code path.

Environment and Infrastructure Status

- Python virtual environment: configured at `.venv`.
- Application dependencies: installed via `requirements.txt`.
- `ffmpeg`: installed and available on host.
- Containers: Redis and Postgres started via `docker-compose.yml`.

Operational Notes

If a health probe returns connection refused, the API process is not running. Restart the service with the project `venv` interpreter and retry `/health`.

Risks and Open Work

- External model availability and model slug drift for provider APIs.
- Need stronger retry, idempotency, and dead-letter handling for production.
- Need dedicated worker process separation and deployment manifests.
- Need observability: tracing, metrics, and per-node latency/cost reporting.

UI Options

- **Streamlit UI** (`ui/streamlit_app.py`): Python-based, simple built-in hot reload, minimal setup.
- **React UI** (`ui/react/`): Modern JavaScript framework, fully customizable styling, Tailwind CSS dark theme, responsive mobile design.
 - Setup: `cd ui/react && npm install && npm start`
 - Runs at `http://localhost:3000`
 - Production build: `npm run build`

Recommended Next Steps

1. Add valid provider keys and run end-to-end quality test with real providers.
2. Introduce separate API and worker services for production runtime.
3. Add retry/backoff policy and failure classification per node.
4. Add subtitle generation and timeline-level QA checks.
5. Add CI checks and integration tests against Redis/Postgres.
6. Deploy React UI to CDN or static hosting (Netlify, Vercel, etc.) for standalone frontend.

Appendix A: Architecture Diagram

